



SMARTLOGIX

Fullstack III

Equipo

Paola Narr
Scarlet Jara
Vicente Avila

Introducción.....	2
Selección de Arquitectura según el caso.....	2
Propuesta de arquitectura de microservicios.....	3
Aplicación de patrones de diseño.....	3
Justificación de estrategias y herramientas.....	4
Flujo general del sistema.....	5
Evaluación del diseño del sistema.....	5
Requerimientos Funcionales.....	5
Requerimientos No Funcionales.....	6
Trazabilidad de Requerimientos.....	7
Diagramas de arquitectura.....	7
Diagrama de contexto.....	7
Diagrama de contenedores.....	7
Diagrama de casos de uso.....	7
Escalabilidad.....	8
Sostenibilidad del sistema.....	8
Seguridad y Privacidad.....	9
Evaluación ética y técnica del diseño.....	9
Conclusión.....	9

Introducción

SmartLogix es una solución tecnológica orientada a optimizar la gestión logística de pequeñas y medianas empresas (PYMEs) de eCommerce. La problemática identificada radica en el uso de sistemas monolíticos y procesos manuales, los cuales generan inconsistencias en inventarios, errores en pedidos y retrasos en envíos.

Para abordar estos desafíos, se propone una arquitectura basada en microservicios, que permite mejorar la escalabilidad, flexibilidad y eficiencia operativa del sistema, facilitando la automatización de procesos logísticos críticos.

Además, esta solución permite a las PYMEs reducir costos operativos, eliminar tareas manuales y mejorar la trazabilidad de sus procesos, lo que resulta clave en entornos de alta demanda y crecimiento constante como el comercio electrónico.

Selección de Arquitectura según el caso

La solución propuesta para SmartLogix se basa en una arquitectura de microservicios, seleccionada en función de las necesidades del caso, donde las PYMEs enfrentan problemas de escalabilidad, sincronización y eficiencia en sistemas monolíticos.

Esta arquitectura permite dividir el sistema en módulos independientes *Inventario*, *Pedidos* y *Envíos*, asegurando una clara separación de responsabilidades. De esta forma, se facilita la adaptación del sistema ante cambios en la demanda, permitiendo escalar componentes específicos sin afectar el funcionamiento global.

Por ejemplo, en eventos de alta demanda como campañas de ventas, el microservicio de Pedidos puede escalar de forma independiente, evitando la sobrecarga del sistema completo, lo cual no sería posible en una arquitectura monolítica.

Además, la arquitectura se alinea con los requerimientos del cliente, ya que permite:

- Automatizar procesos logísticos
- Mejorar la trazabilidad de pedidos
- Optimizar la gestión operativa

Esto convierte a la arquitectura de microservicios en una solución adecuada para entornos dinámicos y de alta demanda como el eCommerce.

Propuesta de arquitectura de microservicios

La arquitectura propuesta sigue un enfoque en capas y desacoplado, estructurando el sistema en componentes claramente definidos:

- Capa de presentación: frontend desarrollado en React.
- Capa de entrada: Traefik como Ingress Controller, encargado del enrutamiento externo.
- Capa de gestión de APIs: KrakenD como API Gateway, responsable de centralizar, filtrar y orquestar las solicitudes.
- Capa de adaptación: Backend for Frontend (BFF), desarrollado en Node.js, que adapta las respuestas según las necesidades del cliente.
- Capa de negocio: microservicios independientes de Inventario, Pedidos y Envíos desarrollados en Spring Boot.
- Capa de datos: bases de datos independientes por servicio (patrón Database per Service).

El diseño incorpora patrones arquitectónicos como API Gateway, BFF y Database per Service, además de patrones de diseño como Repository y Circuit Breaker, permitiendo:

- Desacoplamiento entre componentes
- Escalabilidad independiente por servicio
- Mayor resiliencia ante fallos
- Facilidad de mantenimiento y evolución

El uso del patrón Database per Service permite evitar inconsistencias en los datos, ya que cada microservicio gestiona su propia información de forma independiente, reduciendo conflictos y mejorando la integridad del sistema.

Esta estructura asegura una arquitectura flexible y preparada para entornos de alta demanda.

Aplicación de patrones de diseño

Para asegurar la calidad y mantenibilidad del sistema, se implementan los siguientes patrones:

Repository Pattern: Se utiliza en cada microservicio para gestionar la persistencia de datos de forma desacoplada.

Por ejemplo, en el microservicio de Pedidos, el repositorio permite acceder a la base de datos sin exponer la lógica de acceso, facilitando cambios futuros en la tecnología de persistencia.

Circuit Breaker: Se aplica en la comunicación entre microservicios, especialmente entre Pedidos e Inventario.

Si el servicio de Inventario presenta fallas, el Circuit Breaker evita múltiples intentos fallidos, activando mecanismos alternativos como registrar pedidos en estado pendiente o notificar indisponibilidad temporal.

Esto permite evitar fallos en cascada y mejorar la disponibilidad del sistema.

Factory Method: Se utiliza para la creación de objetos dentro de los microservicios.

Por ejemplo, en la generación de pedidos o envíos, permite instanciar distintos tipos (pedido estándar, express) sin acoplar la lógica de creación, facilitando la extensibilidad del sistema.

En el procesamiento de pedidos, el patrón Repository gestiona el acceso a datos, mientras que el Circuit Breaker protege la comunicación con el microservicio de inventario. A su vez, el Factory Method permite crear distintos tipos de pedidos. Esta combinación permite mantener el sistema desacoplado, resiliente y extensible, asegurando una arquitectura robusta frente a fallos y adaptable a cambios futuros.

Justificación de estrategias y herramientas

Para la implementación de la solución se seleccionaron tecnologías y herramientas en función de criterios de rendimiento, escalabilidad y mantenibilidad.

- React: Se utiliza para construir una interfaz moderna, reutilizable y responsiva, mejorando la experiencia del usuario.
- Spring Boot: Facilita el desarrollo de microservicios robustos y desacoplados mediante APIs REST.
- Node.js (BFF): Permite adaptar las respuestas del backend al frontend, optimizando la comunicación.
- Krakend (API Gateway): Se selecciona por su alto rendimiento y capacidad de centralizar el acceso, reducir llamadas innecesarias y mejorar la seguridad mediante un enfoque declarativo.
- Traefik: Gestiona el enrutamiento y balanceo de carga de manera eficiente.

- JPA: Facilita la persistencia de datos mediante abstracción, mejorando la mantenibilidad del código.

La combinación de estas tecnologías permite construir una solución eficiente, escalable y alineada con buenas prácticas de desarrollo moderno, reduciendo el acoplamiento y mejorando el rendimiento general del sistema.

Flujo general del sistema

El funcionamiento del sistema se inicia cuando un marketplace envía un pedido a SmartLogix. Este es recibido a través del API Gateway, que enruta la solicitud al microservicio de Pedidos.

El microservicio de Pedidos valida la información y consulta al microservicio de Inventario para verificar disponibilidad. Si existe stock suficiente, el pedido es aprobado y registrado en la base de datos.

Posteriormente, se genera una solicitud al sistema de transportistas para coordinar el envío, mientras que el sistema de pagos confirma la transacción correspondiente.

Finalmente, el estado del pedido es actualizado y comunicado a los sistemas externos, asegurando trazabilidad completa del proceso logístico.

Evaluación del diseño del sistema

El diseño propuesto cumple con los requerimientos funcionales definidos para el sistema, permitiendo:

- Gestión de inventario en tiempo real
- Procesamiento automatizado de pedidos
- Coordinación eficiente de envíos

Requerimientos Funcionales

El sistema contempla funcionalidades clave organizadas por módulos, permitiendo automatizar y optimizar los procesos logísticos de las PYMEs:

Inventario

- Registro y actualización de productos en tiempo real.
- Sincronización de stock entre múltiples bodegas y tiendas.
- Generación de alertas automáticas ante niveles bajos de inventario.

Pedidos

- Registro automático de pedidos provenientes de marketplaces.
- Validación de pedidos en función de la disponibilidad de stock.
- Aprobación automatizada de pedidos y asignación de estado.
- Trazabilidad completa del pedido durante todo su ciclo de vida.

Envíos

- Generación automática de órdenes de envío a partir de pedidos aprobados.
- Asignación de transportistas según disponibilidad y ubicación.
- Seguimiento en tiempo real del estado de entrega.

Plataforma

- Autenticación y autorización de usuarios mediante tokens JWT.
- Integración con sistemas externos como marketplaces, pasarelas de pago y transportistas.
- Comunicación entre servicios mediante APIs REST.

Requerimientos No Funcionales

El sistema asegura calidad y rendimiento mediante los siguientes atributos:

- Rendimiento: Tiempo de respuesta menor a 2 segundos por solicitud.
- Escalabilidad: Capacidad de escalar microservicios de forma independiente según la demanda.
- Seguridad: Implementación de autenticación mediante JWT y comunicación segura mediante HTTPS.
- Disponibilidad: Tolerancia a fallos mediante el uso del patrón Circuit Breaker.
- Calidad: Implementación de pruebas unitarias con cobertura mínima del 60%.
- Mantenibilidad: Arquitectura desacoplada que facilita la evolución del sistema.

Estos requerimientos no funcionales influyen directamente en las decisiones arquitectónicas, justificando el uso de microservicios, API Gateway y patrones de resiliencia para garantizar el correcto funcionamiento del sistema.

Trazabilidad de Requerimientos

Los requerimientos definidos se relacionan directamente con la arquitectura propuesta, asegurando su correcta implementación:

- La gestión de inventario en tiempo real se implementa mediante el microservicio de Inventario y su base de datos independiente.
- El procesamiento automatizado de pedidos se gestiona a través del microservicio de Pedidos, integrando validación y trazabilidad.
- La coordinación de envíos se realiza mediante el microservicio de Envíos, conectado con sistemas de transportistas externos.
- La seguridad del sistema se implementa mediante autenticación basada en JWT y control de acceso.
- La disponibilidad y resiliencia se aseguran mediante el uso del patrón Circuit Breaker en la comunicación entre servicios.

Diagramas de arquitectura

Para representar la solución se utilizan:

Diagrama de contexto

El diagrama de contexto representa la interacción de SmartLogix con sistemas externos como marketplaces, sistemas de pago y transportistas. Estos sistemas permiten automatizar la recepción de pedidos, validar transacciones y coordinar envíos, eliminando procesos manuales y mejorando la eficiencia operativa.

Diagrama de Contexto

Diagrama de contenedores

Representa la estructura interna del sistema, incluyendo frontend, API Gateway, BFF y microservicios. Permite visualizar cómo se distribuyen las responsabilidades y cómo interactúan los componentes.

Diagrama de Contenedores

Diagrama de casos de uso

El diagrama de casos de uso representa la interacción entre los actores del sistema SmartLogix y sus principales funcionalidades, abarcando los módulos de inventario, pedidos y envíos.

Permite visualizar cómo los sistemas externos y usuarios internos participan en los procesos logísticos. Además, modela tanto flujos obligatorios como escenarios excepcionales mediante relaciones <<include>> y <<extend>>.

Diagrama de Casos de Uso

Estos diagramas permiten validar que la arquitectura cumple con los requerimientos de escalabilidad, desacoplamiento y resiliencia definidos en el caso.

Escalabilidad

La arquitectura permite escalar horizontalmente los microservicios de forma independiente.

Por ejemplo:

- El microservicio de Pedidos puede escalar en momentos de alta demanda sin afectar Inventario o Envíos.

El uso de contenedores (Docker) y orquestadores (Kubernetes) permite:

- Automatizar despliegues
- Balancear carga
- Asegurar alta disponibilidad

El API Gateway permite distribuir el tráfico eficientemente, evitando cuellos de botella.

Además, la arquitectura permite integrar nuevos servicios como sistemas de facturación o analítica sin afectar los existentes, asegurando su crecimiento a futuro.

Sostenibilidad del sistema

La arquitectura basada en microservicios permite una evolución continua del sistema, facilitando la incorporación de nuevas funcionalidades sin afectar los servicios existentes. Permite escalar solo lo necesario. Si hay muchos pedidos pero poco inventario, sólo escalas el MS de Pedidos, ahorrando recursos de cómputo y energía.

También permite:

- Reducir costos operativos al escalar solo componentes necesarios
- Minimizar el impacto de fallos
- Mantener el sistema en el tiempo con menor esfuerzo

Esto asegura la sostenibilidad técnica y operativa a largo plazo.

Seguridad y Privacidad

El API Gateway actúa como barrera de seguridad (autenticación/autorización) y las bases de datos están en una red privada (aisladas del exterior).

Al usar microservicios, los datos sensibles (como pagos) pueden ser aislados y cifrados de forma independiente.

Se implementan mecanismos de control de acceso y auditoría que permiten registrar acciones dentro del sistema, reduciendo riesgos de uso indebido de la información.

Evaluación ética y técnica del diseño

La solución considera aspectos éticos y técnicos relevantes:

- Protección de datos mediante mecanismos de seguridad
- Uso responsable de la información de clientes
- Transparencia en los procesos logísticos

Además, la arquitectura se alinea con las necesidades del cliente, proporcionando una solución eficiente, accesible y escalable para PYMEs.

Conclusión

La plataforma SmartLogix representa una solución moderna basada en microservicios que responde a los desafíos del comercio electrónico actual. La correcta selección de arquitectura, herramientas y patrones de diseño permite garantizar un sistema escalable, eficiente y mantenible.

En conclusión, la propuesta cumple con los principios de diseño desacoplado, resiliente y orientado a servicios, asegurando su viabilidad en entornos reales de alta demanda.